



© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2023

M. Frisbie, *Building Browser Extensions*

https://doi-org.ezproxy.sfpl.org/10.1007/978-1-4842-8725-5_7

7. Background Scripts

Matt Frisbie¹

(1) California, CA, USA

Browser extension developers often find the need to execute JavaScript in the background of a browser. Background scripts, as their name suggests, are the answer to this problem: they are a separate JavaScript runtime that can communicate with all the different pieces of the extension, add handlers for many different browser events, and operate without dependence on any web page or extension user interface.

Note This chapter will focus on the manifest v3 implementation of background scripts as service workers. Manifest v2 featured background pages, which were considerably more useful. However, Google Chrome is phasing out support for them, so I do not recommend building a browser extension that explicitly relies on manifest v2 background pages.

Web Page Service Workers vs. Extension Service Workers

A common point of confusion for web developers new to browser extensions is the nature of background service workers. Although many web developers have experience dealing with service workers in the context of building web pages, the nature of service workers in extensions has some notable differences.

Compare the following two examples, a simple web page service worker and a simple browser extension service worker:

Web page service worker

```
// Establish a cache name
const cacheName = 'cache_v1';
// Assets to precache
const precachedAssets = [
  '/img1.jpg',
  '/img2.jpg',
  '/img3.jpg'
];
self.addEventListener('install', (event) => {
  // Precache assets on install
  event.waitUntil(caches.open(cacheName).then((cache) => {
    return cache.addAll(precachedAssets);
  }));
});
self.addEventListener('fetch', (event) => {
  // Test for precached assets match
  const url = new URL(event.request.url);
  const isPrecachedRequest =
    precachedAssets.includes(url.pathname);
  if (isPrecachedRequest) {
    // Grab the precached asset from the cache
    event.respondWith(caches.open(cacheName).then((cache) => {
      return cache.match(event.request.url);
    }));
  } else {
    // Go to the network
    return;
  }
});
```

This simple web page service worker defines a cache for some images. On an install event, it preloads the images into the cache. When it sees

a fetch event matching the URL for one of those images, it intercepts the request and returns the value from the cache.

Extension service worker

```
// This will run on an install or update
chrome.runtime.onInstalled.addListener(() => {
  console.log('Installed or updated!');
});
// This will run when a bookmark is created.
chrome.bookmarks.onCreated.addListener(() => {
  console.log('Created a new bookmark!');
});
const filter = {
  url: [
    {
      urlMatches: 'https://www.example.com/',
    },
  ],
};
// This will run when the example site is visited
chrome.webNavigation.onCompleted.addListener(() => {
  console.log("Loaded the example site!");
}, filter);
```

This simple extension service worker sets listeners for the `runtime.onInstalled`, `bookmark.onCreated`, and `webNavigation.onCompleted` events. Each handler prints a log statement to the console when those events are fired.

Similarities

The W3C specification for service workers begins with the following high-level characterization:

The core of this specification is a worker that wakes to receive events.

In this respect, a web page service worker and an extension service worker are serving the same purpose. What's more, the distinction between a "web page" service worker and an "extension" service worker only describes how the service worker is deployed in that context; the underlying service worker platform remains unchanged.

When comparing the two examples from above, you may have noticed the following:

- Both service worker scripts are largely composed of event handlers being assigned in the top level of the script.
- Both service workers are listening for some form of an installation event.
- Both service workers can access network requests occurring in the browser.

These similarities underscore many of the similarities between the two. With these code similarities in mind, let's explore the ways that these service worker contexts are alike.

Single Occupancy

For both web page service workers and extension service workers, there will only ever be a single service worker per script. When installing updates to service workers, the browser will carefully swap out old service workers for new ones while ensuring there is only ever a single service worker active at any given time.

This is critically important in both contexts: multiple service workers would cause complete chaos with caching, network interception, and event handling.

Installation and Lifecycle

All service workers are installed exactly once, and subsequently may become idle and eventually terminated. The browser will decide when

to wake the service worker, usually because of an event being fired, and the service worker script will re-execute each time.

Although the concept of an “installed” event has different meanings for web pages and extensions, the use of these handlers remains unchanged. The developer should expect that the service worker script will be executed an unlimited number of times, but the installed handler will execute only once and therefore should be utilized to perform setup work that should not be duplicated.

Top-Level Event Handling

Because the service worker is expected to undergo a repetitive cycle of becoming idle, being terminated, and being re-activated, service worker script structures must account for this. Events that are fired by the browser and passed to the service worker must have handlers added in the first turn of the event loop – otherwise there is a risk of missing the event. The reason for this is that service workers are often woken in direct response to an event being fired. The browser will wake the service worker, execute one turn of the event loop, and then fire the queued event in the service worker. If the handler for that event is not present at the end of that turn, the event may pass through unhandled.

Async Messaging

Because all service workers are strictly asynchronous, both must use a form of asynchronous messaging to communicate with other parts of the browser.

- For web page service workers, this takes the form of `postMessage()` or the `MessageChannel` API.
- For browser extensions, this takes the form of `runtime.sendMessage()`, `tabs.sendMessage()`, `runtime.connect()`, or `tabs.connect()`.

Differences

Although web page service workers and extension service workers use the same underlying platform, they differ in a handful of important ways.

Registration

Web page service workers must be registered from a page-level script. An example of this is shown below:

Example web page service worker registration

```
if ('serviceWorker' in navigator) {  
  navigator.serviceWorker.register('/sw.js')  
    .then((registration) => {  
    registration.addEventListener('updatefound', () => {  
      console.log('A new service worker is being installed:');  
    });  
  })  
    .catch((error) => {  
      console.error(`Service worker registration failed: ${error}`);  
    });  
} else {  
  console.error('Service workers are not supported.');
```

For extension service workers, the only step needed to register a service worker is to specify the `background.service_worker` script in the manifest. Web page service worker registration includes the `installing/waiting/active` states that are not meaningful for extension service workers.

Note For additional information on web page service workers, refer to https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API.

Purpose

The most important difference between web page service workers and extension service workers is their primary purpose. Web page service workers are equipped to act as a cache, and this is their most common use case. They're able to conditionally intercept network requests and return cached content. Service workers can also be used to build progressive web applications (PWAs), web pages that emulate app behavior with installation, offline capabilities, and server-sent push notifications.

Because browser extension scripts are assets and all served from the browser, there is no longer a need to cache these resources, as they are not being loaded from a remote server. Furthermore, interfaces such as popup pages and options pages are perfectly capable of operating normally offline without the use of a background service worker. Instead, extension service workers are primarily tasked with handling the wide range of events that are fired by the browser and the WebExtensions API.

NoteFor the remainder of this chapter, an extension service worker will be simply referred to as a “service worker.”

Manifest v2 vs. Manifest v3

Manifest v3 transformed background scripts considerably. Manifest v2 background scripts were treated as *background pages*: JavaScript that executed in a headless web page. These background pages were also capable of running indefinitely, meaning they were suitable for managing long-running operations and web requests. In manifest v3, background scripts now execute as *service workers*. These workers are more lightweight, but they are diminished in a number of important ways.

Scripts vs. Service Workers

In manifest v2, the manifest background value could be passed an array of scripts:

Manifest v2 manifest.json

```
{
  "manifest_version": 2,
  ...
  "background": {
    "scripts": ["bg1.js", "bg2.js", "bg3.js"]
  }
  ...
}
```

These scripts were loaded in the order they appeared in the array, and all executed inside the same background page. In manifest v3, the background script is a single service worker:

Manifest v3 manifest.json

```
{
  "manifest_version": 3,
  ...
  "background": {
    "service_worker": "bg.js"
  }
  ...
}
```

JavaScript Imports

In manifest v2, the only way to use a JavaScript `import` was to use the `background.page` property and to use script tags in the background page HTML:

manifest.json enabling a background HTML page


```
{
  "manifest_version": 2,
  ...
  "background": {
    "page": "bg.html"
  }
  ...
}
```

bg.html

```
<html>
  <body>
    <!-- imports allowed in this JS file -->
    <script type="module" src="bg.js"></script>
  </body>
</html>
```

In manifest v3, you can simply set the `background.module` property to `"module,"` shown here:

Manifest v3 manifest.json allowing for the `import` keyword

```
{
  "manifest_version": 3,
  ...
  "background": {
    "service_worker": "bg.js",
    "type": "module"
  }
  ...
}
```

No Access to DOM and Limited Global APIs

In manifest v2, background pages are effectively headless web pages with full access to a DOM and a web page global object. In manifest v3, service workers

have no access to a DOM and the global object is the `ServiceWorkerGlobalScope`, which is missing many of the APIs available in the web page global object. This has some important implications:

- **No access to the `document` object.** The DOM and all its associated methods are no longer available. This also means the background service worker can no longer create and cache assets.
- **No ability to render content for display in other contexts.** Rendering content in the headless page was useful in some situations, but this is no longer possible. Partial workarounds include using a third-party library like `jsdom` or using the `OffscreenCanvas` API.
- **No access to the `window` object.** This means that methods like `window.open()` are no longer available.
- **No ability to play or capture media directly in a service worker.** Doing so is still possible, but it requires a host active content script or an extension view to grant access to the media APIs.
- **No access to `localStorage`, `sessionStorage`, cookies.**
- **No ability to use `XMLHttpRequest`.** `fetch()` should be used.

Note Losing access to the `window` object is especially problematic when attempting to authenticate from the service worker, as third-party authentication libraries frequently rely on opening a new window for the OAuth flow. Refer to the *Networking* chapter for more coverage on authentication.

Nonpersistent

Manifest v2 allowed for background pages to be “persistent,” which effectively meant they would run indefinitely as long as the host browser program was open. In manifest v3, this ability is removed. This has several implications:

- **`setTimeout()` and `setInterval()` will no longer reliably execute.** These methods are still available. However, if the service worker terminates before the scheduled function executes, it will silently be skipped. The `chrome.alarms` API replaces some of this

functionality, but it cannot be used for time intervals less than a minute.

- **Long-running queries and websockets may be killed.** The service worker will ignore open network connections when determining if a service worker is idle, meaning that these connections may be severed if the service worker terminates while the connections are still active.

Tip There are some tricks later on in this chapter that can extend the life of a service worker.

No Shutdown Event

Manifest v2 allowed the developer to add handler for `runtime.onSuspend` and `runtime.onSuspendCanceled` events. This gave the extension opportunity to do some clean up. In manifest v3, service workers have no handler for a shutdown event, meaning that service worker scripts must eagerly perform tasks with the expectation that a shutdown could occur at any time.

No Programmatic Background Access

In manifest v2, it was possible to access the background window object from an extension interface in the foreground using `chrome.extension.getBackgroundPage()`. In manifest v3, this ability has been removed.

Working with Background Scripts

Let's begin by creating a simple browser extension with a background script. This is accomplished by defining the `background.service_worker` property in the manifest:

```
{  
  "name": "MVX",
```

```
"version": "0.0.1",
"manifest_version": 3,
"background": {
  "service_worker": "background.js"
}
}
```

Example 7-1a manifest.json

```
console.log("Initialized background script!");
chrome.runtime.onInstalled.addListener((object) => {
  console.log("Installed background script!");
});
```

Example 7-1b background.js

This simple background script logs to the console, sets an install handler, and exits.

Inspecting Background Service Workers

After installing this extension in your browser, your extension management page will show a new card for the extension that includes a *service worker* link (Figure [7-1](#)).

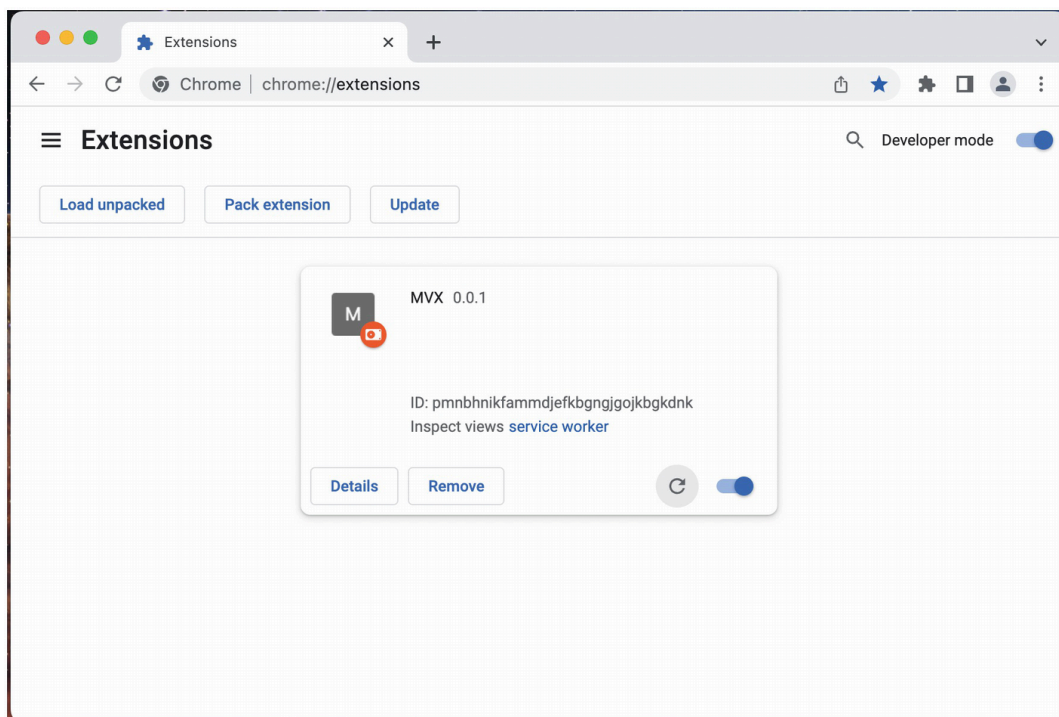


Figure 7-1 The installed extension with background service worker

You can inspect the background script by clicking the *service worker* link (Figure [7-2](#)).

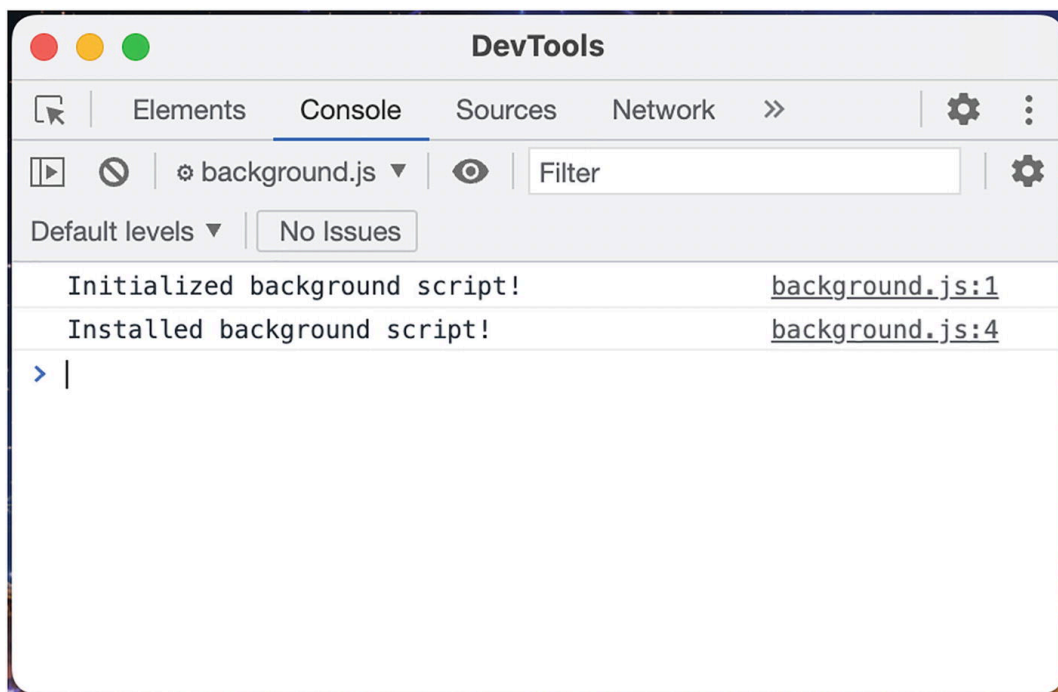


Figure 7-2 The console output of the background service worker

Note This devtools interface is very useful for inspecting and debugging your service worker, but keep in mind that leaving this window open will prevent the browser from suspending the service worker. Unintentionally leaving a devtools window open can cause unexpected extension behavior when attempting to reload or debug background scripts.

To view the state of all service workers in the browser without interfering with their normal lifecycle, navigate your browser to `chrome://serviceworker-internals/`, shown in Figure [7-3](#).

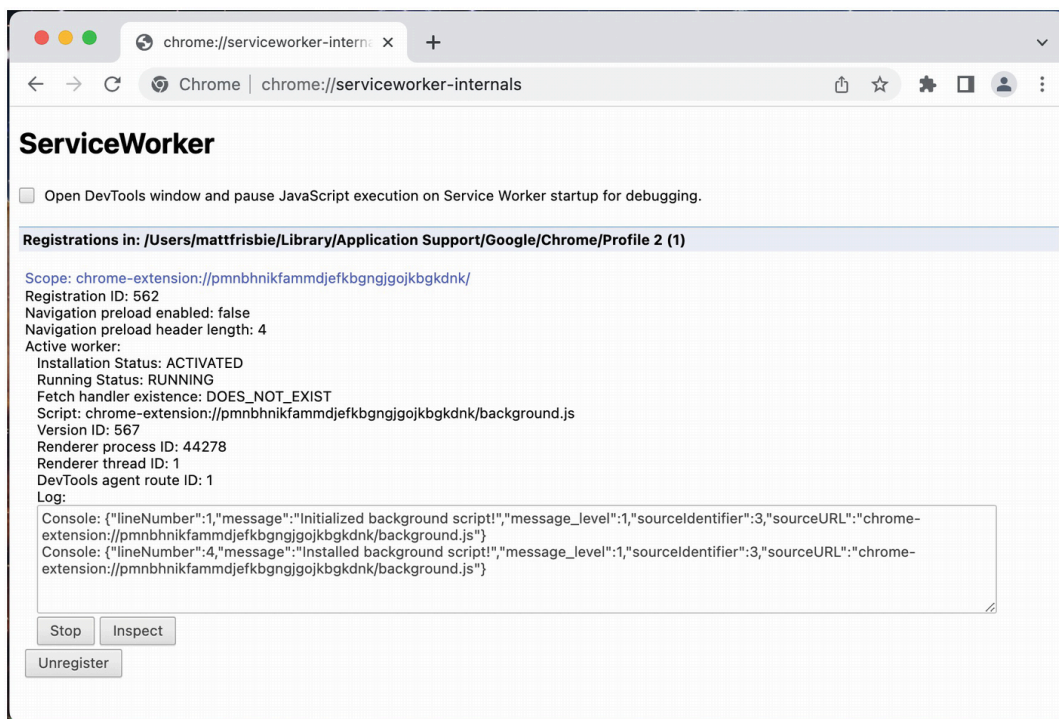


Figure 7-3 Chrome's service worker internals page

Note This page will show *all* service workers your browser has installed – not just for extensions. Recall that the browser does not make a distinction between a web page service worker and an extension service worker.

Service Worker Errors

Note For this section, you must begin with the extension installed as shown in Example 7-1 and all devtools windows closed.

Modify the background script as shown here:

```
console.log("Initialized background script!");
chrome.runtime.onInstalled.addListener((object) => {
  console.log("Installed background script!");
});
throw new Error("foo");
```

Example 7-2 background.js

Once the background script is saved, click the reload button in the extension card. You will see the service worker immediately becomes inactive

and an *Errors* button appears (Figure 7-4).

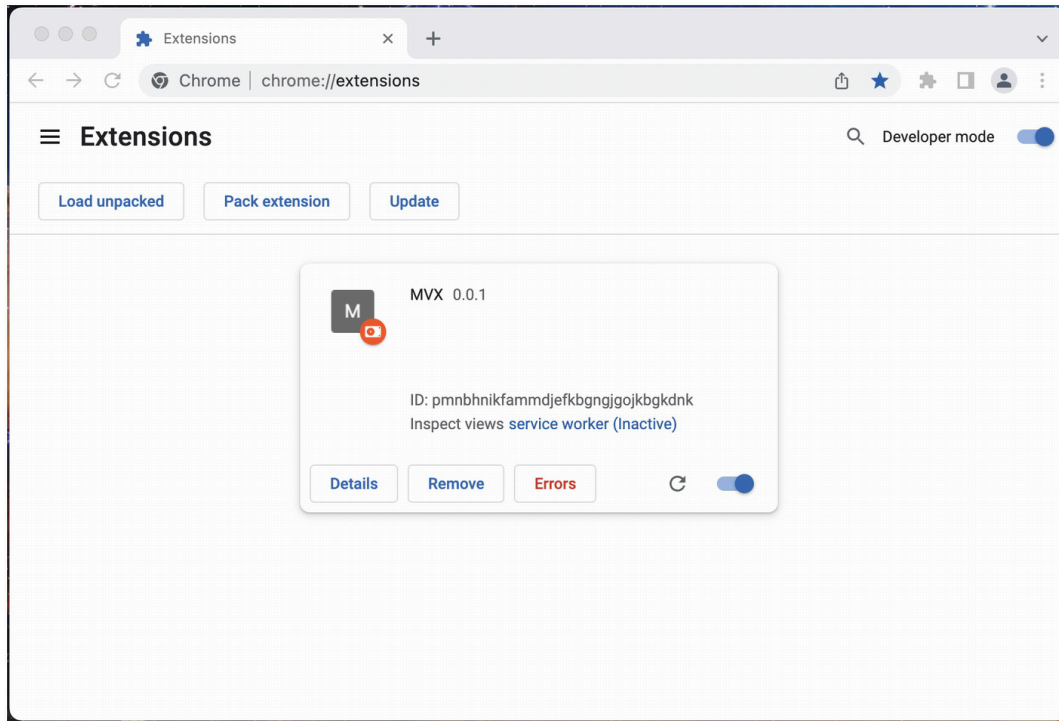


Figure 7-4 The extension Errors button appears

Click the Errors button to reveal the error view (Figure 7-5).

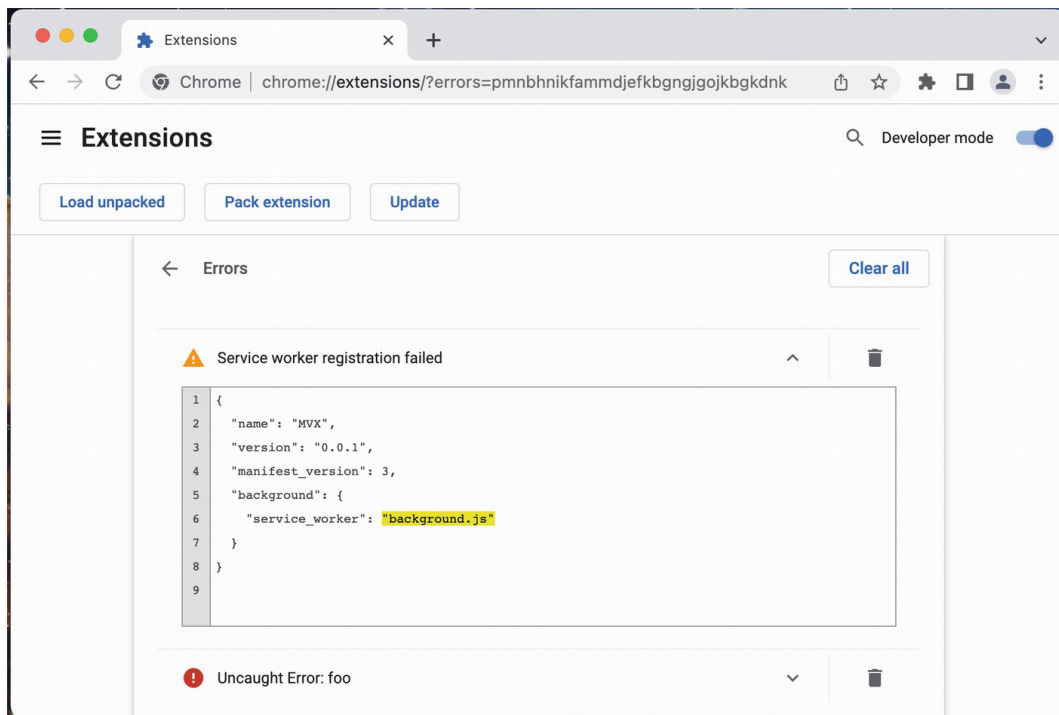


Figure 7-5 The extension error view

You will see the expected “foo” error readout at the bottom, but also that the browser is telling us that the service worker failed to register. This is very important: **when a service worker throws an error in the first turn of the event loop, the service worker will fail to register.**

Note The errors page is especially useful in situations where the background script fails to register. When registration fails, since the service worker does not exist, you will not be able to inspect the error output of the service worker. Instead, the errors page will contain the error readout to assist in debugging.

Service Worker Termination

Note For this section, you must keep all devtools windows closed to allow the service worker to become idle.

Install the following extension:

```
{
  "name": "MVX",
  "version": "0.0.1",
  "manifest_version": 3,
  "action": {},
  "background": {
    "service_worker": "background.js"
  }
}
```

Example 7-3a manifest.json

```
console.log("Initialized background script!");
chrome.runtime.onInstalled.addListener((object) => {
  console.log("Installed background script!");
});
chrome.action.onClicked.addListener(() => {
  console.log("Clicked toolbar icon!");
});
```



```
let elapsed = 0;
setInterval(() => console.log(`${++elapsed}s`), 1000);
```

Example 7-3b background.js

This background script will log to the console every 1000ms until the service worker is terminated. (Recall that `setInterval()` will not prevent a service worker from becoming idle.) It will also log to the console whenever the toolbar icon is clicked.

After reloading the extension, open the `chrome://serviceworker-internals` page to observe the log output. The browser sees that this extension is not actively doing work, and therefore is considered to be *idle*. After approximately 30 seconds, you will see the browser stop the service worker (Figure 7-6).

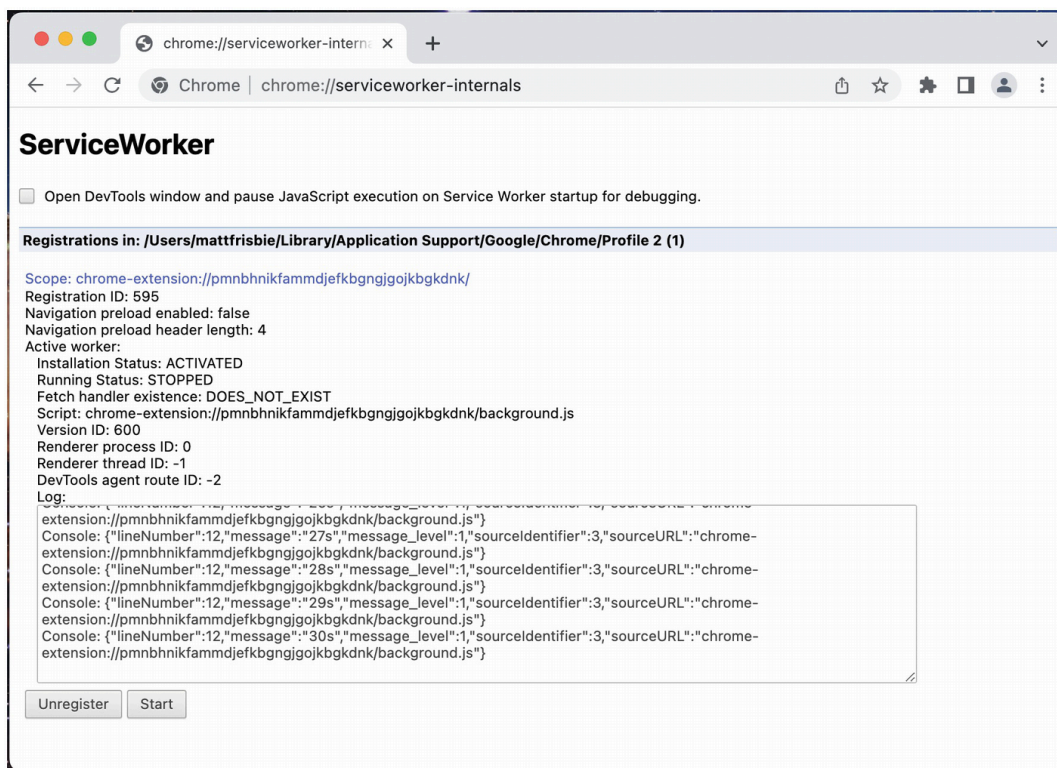


Figure 7-6 The browser stops the service worker after being idle for 30 seconds

The extension management page will also reflect this (Figure 7-7).

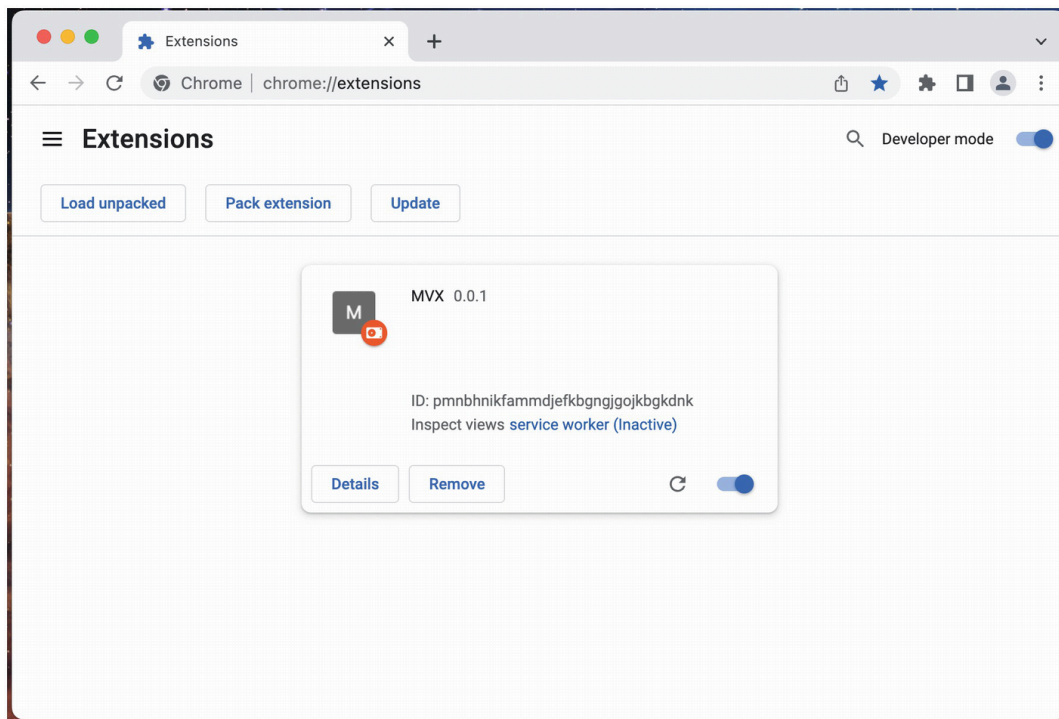


Figure 7-7 The inactive service worker as shown on the extension management page

Because you set a handler for the `action.onClicked`, the browser will wake the service worker, rerun the script, and restart the counter once you click the toolbar icon.

Note Carefully observe the console output after clicking the toolbar icon button. You will notice that the global state in the service worker has been reset.

Common Patterns

In this section, we will explore a handful of patterns that background service workers are frequently used for.

Event Handler

The WebExtensions API allows the browser to send a huge range of different events into the extension to be handled. Since the API is accessible in content scripts, popups, options pages, and background scripts, these events can technically be handled in any of these places. However, since content scripts, popups, and options pages are ephemeral, only the background script

guarantees that the event will be handled. The following example extension sets up a background script to handle a myriad of different API events:

```
{
  "name": "MVX",
  "version": "0.0.1",
  "manifest_version": 3,
  "background": {
    "service_worker": "background.js"
  },
  "permissions": ["alarms", "tabs"],
  "action": {},
  "commands": {
    "foobar": {
      "suggested_key": {
        "default": "Ctrl+Shift+J",
        "mac": "MacCtrl+Shift+J"
      },
      "description": "Perform foobar action"
    }
  }
}
```

Example 7-4a manifest.json

```
// Logs when the extension is installed
chrome.runtime.onInstalled.addListener(() => {
  console.log("Installed background!");
});
chrome.alarms.create("My alarm", { periodInMinutes: 1 });
// Logs every minute
chrome.alarms.onAlarm.addListener((alarmInfo) => {
  console.log(`Alarm fired: ${alarmInfo.name}`);
});
// Logs when the tab state changes
chrome.tabs.onUpdated.addListener(() => {
  console.log("Tabs updated");
});
// Logs when Ctrl+Shift+J is typed
chrome.commands.onCommand.addListener((command) => {
```

```
    console.log(`Command: ${command}`);  
  });  
  // Logs when the toolbar icon is clicked  
  chrome.action.onClicked.addListener(() => {  
    console.log(`Clicked toolbar icon`);  
  });  
});
```

Example 7-4b background.json

Of course, this code could be transplanted into a popup script as-is. However, the events would *only* be handled if the popup was open. Placing handlers in the background script guarantees that the handlers will execute.

If the service worker is terminated, the browser will restart the service worker, run one turn of the event loop to set all the handlers, and only then will it fire all the events inside the service worker. In this way, you can guarantee event handling by assigning handlers at the top level.

Secret Management and Authentication

Extensions frequently need to manage authentication and secret values apart from untrusted web pages. This is especially true in situations where a content script needs to communicate with a server. Sending the request from the content script will usually not work, as the content script's network requests are still bound by the host page's cross-origin restrictions. Furthermore, collecting credentials or storing the authentication secrets in the content script is a *terrible* idea, as the page's DOM and storage APIs are shared by the host page:

- If a content script were to place a JSON Web Token (JWT) into `localStorage`, the host page's JavaScript could access that JWT.
- If a content script collects credentials in `<input>` tags in the host page, the host page is capable of reading the values out of those HTML elements.

The following is an example of a secure authentication flow:

1. Collect the user's credentials in a trusted user interface, such as a popup page or options page. (Anything with the host `chrome-extension://my-extension-id` or similar).
2. Authenticate with the credentials from the trusted page.
3. Store the authentication token with the `chrome.storage` API

After performing these steps, your background service worker can access this authentication token and send authenticated requests to the remote server. A content script can indirectly send authenticated requests via a message relay:

1. Content script uses extension message API to send a message to the background indicating the type of authenticated request it wishes to send
2. Background script accesses the authentication token from storage and sends the authenticated request
3. When the request returns, the background script collects the response and sends it back up to the requesting content script
4. Content script receives the response payload via the extension message API

This strategy is advantageous for several reasons:

- The content script never has to collect credentials
- The content script never has to store secrets
- The content script is no longer bound by the host page's cross-origin restrictions

Message Hub

Any piece of an extension can use the extension messaging API, but the background script is the only entity that guarantees the message will be heard. Furthermore, it also guarantees that the message will be received exactly once. In this way, it is often useful for an array of content scripts to all connect to a single background service worker and use it as a message hub. The following example extension will create a message port on each page that

is opened and repeatedly send messages to the background. Install the extension, open several tabs of <https://example.com>, and inspect the logs of the pages and the background service worker to observe the background independently communicating with each page.

```
{
  "name": "MVX",
  "version": "0.0.1",
  "manifest_version": 3,
  "background": {
    "service_worker": "background.js",
    "type": "module"
  },
  "content_scripts": [
    {
      "matches": ["<all_urls>"],
      "css": [],
      "js": ["content-script.js"]
    }
  ]
}
```

Example 7-5a manifest.json

```
// Messages include information about the sender
chrome.runtime.onMessage.addListener((request, sender, sendResponse) => {
  // This handler will fire for all runtime.sendMessage payloads.
  // Use a value in the payload to filter.
  if (request.type === "getCurrentTabId") {
    // Send the tab's ID back to the content script
    sendResponse({ currentTabId: sender.tab.id });
  }
});
// This will fire when the content script
// calls runtime.connect()
chrome.runtime.onConnect.addListener((port) => {
  console.log(`Connected to ${port.name}`);
  // Messages sent from the content script
  port.onMessage.addListener((msg) => {
    console.log(port.name, msg);
  });
});
```

```
// Subtract 1 and send value back up to content script
port.postMessage({ value: msg.value - 1 });
});
});
```

Example 7-5b background.js

```
function initializeCountdown(currentTabId) {
  // This will fire the runtime.onConnect event
  // in the background
  const port = chrome.runtime.connect({
    name: `Tab ${currentTabId}`,
  });
  // Messages sent from the service worker
  port.onMessage.addListener((msg) => {
    console.log(port.name, msg);
    // Keep passing the value to the background while > 0
    if (msg.value > 0) {
      port.postMessage({ value: msg.value });
    }
  });
  // Start a countdown sequence every 1000ms
  setInterval(() => {
    const value = Math.floor(Math.random() * 10) + 1;
    console.log(`New countdown sequence: ${value}`);
    // Send the initial postMessage in the sequence
    port.postMessage({ value });
  }, 1000);
}
// Send a call/response message to the background
// to determine current tab's ID
chrome.runtime.sendMessage(
  // Providing a type allows the background to filter
  // incoming messages
  { type: "getCurrentTabId" },
  // Background can reply to this message with the tab ID
  (response) => initializeCountdown(response.currentTabId)
);
```

Example 7-5c content-script.js

Storage Manager

For extensions that need to store large amounts of data, the background service worker can make use of IndexedDB. Popups, options pages, and content scripts can all use the messaging API to read and write to IndexedDB. The advantage of using the background as the IndexedDB manager is versioning: the browser guarantees one background service worker per extension at any time, so any IndexedDB migrations you need to run can safely execute inside the service worker without risking a versioning conflict.

Injecting Scripts

Background service workers can programmatically inject content scripts into the page. This is very useful when you need to inject a content script conditionally or asynchronously. The following example injects a small content script when the toolbar icon button is clicked:

```
{
  "name": "MVX",
  "version": "0.0.1",
  "manifest_version": 3,
  "background": {
    "service_worker": "background.js",
    "type": "module"
  },
  "action": {},
  "permissions": ["activeTab", "scripting"]
}
```

Example 7-6a manifest.json

```
function wipeOutPage() {
  document.body.innerHTML = "";
}
chrome.action.onClicked.addListener((tab) => {
  chrome.scripting.executeScript({
    target: { tabId: tab.id },
```



```
func: wipeOutPage,  
});  
});
```

Example 7-6b background.js

Note Refer to the *Content Scripts* chapter for more details on this.

Sniffing Web Traffic

The background service worker can track the pages the user is visiting and conditionally perform logic based on the details of that page. The following example logs the URL on every page visited, and logs a special message only on a specific URL:

```
{  
  "name": "MVX",  
  "version": "0.0.1",  
  "manifest_version": 3,  
  "background": {  
    "service_worker": "background.js",  
    "type": "module"  
  },  
  "action": {},  
  "permissions": ["webNavigation"]  
}
```

Example 7-7a manifest.json

```
const filter = {  
  url: [  
    {  
      urlMatches: "https://www.example.com/",  
    },  
  ],  
};  
  
chrome.webNavigation.onCompleted.addListener(() => {  
  console.log("Visited the special site!");  
}, filter);  
  
chrome.webNavigation.onDOMContentLoaded.addListener((details) => {
```

```
console.log(`Loaded ${details.url}!`);  
});
```

Example 7-7b background.js

Installed/Updated Events

Developers often need to run a piece of code only when the extension is updated or installed. The `runtime.onInstalled` event should be used for this purpose. It is capable of differentiating between install events and different update events via the `reason` property. The following background script identifies when each install reason will execute:

background.js

```
console.log("This will run each time the service worker starts");  
chrome.runtime.onInstalled.addListener((details) => {  
  switch (details.reason) {  
    case chrome.runtime.OnInstalledReason.INSTALL:  
      console.log("This runs when the extension is newly installed.");  
      break;  
    case chrome.runtime.OnInstalledReason.CHROME_UPDATE:  
      console.log("This runs when a chrome chrome update installs.");  
      break;  
    case chrome.runtime.OnInstalledReason.SHARED_MODULE_UPDATE:  
      console.log("This runs when a shared module update installs.");  
      break;  
    case chrome.runtime.OnInstalledReason.UPDATE:  
      console.log("This runs when an extension update installs.");  
      break;  
    default:  
      break;  
  }  
});
```

Opening Tabs

Content script can't open extension URLs, but they can delegate the opening to the background script. The following example adds two buttons to the page: one will fail to open the extension URL indirectly, and one sends a message to the background script to open the URL.

```
{
  "name": "MVX",
  "version": "0.0.1",
  "manifest_version": 3,
  "background": {
    "service_worker": "background.js",
    "type": "module"
  },
  "content_scripts": [
    {
      "matches": ["<all_urls>"],
      "css": [],
      "js": ["content-script.js"]
    }
  ],
  "permissions": ["tabs"]
}
```

Example 7-8a manifest.json

```
chrome.runtime.onMessage.addListener((msg) => {
  chrome.tabs.create({ url: msg.url });
});
```

Example 7-8b background.js

```
const root = document.createElement("div");
root.innerHTML = `
<div style="position:fixed;top:0;left:0">
  <button id="direct-open">THIS WILL NOT WORK</button>
  <button id="indirect-open">THIS WILL WORK</button>
</div>
`;
document.body.appendChild(root);
const url = chrome.runtime.getURL("foobar.html");
```

```
document.querySelector("#direct-open").addEventListener("click", () => {  
    window.open(url);  
});  
document.querySelector("#indirect-open").addEventListener("click", () => {  
    chrome.runtime.sendMessage({ url });  
});
```

Example 7-8c content-script.js

```
<!DOCTYPE html>  
<html>  
  <body>  
    <h1>Foobar</h1>  
  </body>  
</html>
```

Example 7-8d foobar.html

Forcing Service Worker Persistence

As described earlier in the chapter, service workers are designed to shut down frequently. For most extension developers, this is not an issue, as their needs can be adequately met by the service worker form factor. For the handful of situations where a persistent background script is *absolutely essential*, there are no officially supported remedies. However, there are some unofficial strategies that can prolong the life of your service worker.

Implementation

All the forced persistence workarounds all rely on the same overall strategy. When an extension message port is opened between a content script and the background, the browser will not terminate the service worker for 5 minutes. There does not need to be any message passing going on, the port just needs to remain open. Once these 5 minutes are up, the browser may decide the service worker is idle and terminate it. If, however, a new port is connected before 5 minutes is

up, the browser will restart the 5-minute timer. If the extension continually re-opens new ports, the background service worker can theoretically remain active indefinitely.

Stack overflow user wOxxOm has an excellent writeup about this here: <https://stackoverflow.com/questions/66618136/persistent-service-worker-in-chrome-extension#answer-66618269>. One of their demos is adapted below:

```
{
  "name": "MVX",
  "version": "0.0.1",
  "manifest_version": 3,
  "background": {
    "service_worker": "background.js",
    "type": "module"
  },
  "permissions": ["scripting"],
  "host_permissions": ["<all_urls>"]
}
```

Example 7-9a manifest.json

```
let lifeline;
// Disconnect and reconnect
function keepAliveForced() {
  lifeline?.disconnect();
  lifeline = null;
  keepAlive();
}
async function keepAlive() {
  if (lifeline) {
    return;
  }
  // Locate any eligible tab and connect to it
  for (const tab of await chrome.tabs.query({})) {
    try {
      await chrome.scripting.executeScript({
        target: { tabId: tab.id },
```

```

    function: () => chrome.runtime.connect({
      name: "KEEPALIVE"
    }),
  });
  return;
} catch (e) {}
}
}
chrome.runtime.onConnect.addListener((port) => {
  if (port.name == "KEEPALIVE") {
    lifeline = port;
    // Refresh the connection after 1 minute
    setTimeout(keepAliveForced, 6e4);
    port.onDisconnect.addListener(keepAliveForced);
  }
});
// Any tab change means reconnecting may be required
chrome.tabs.onUpdated.addListener((tabId, info, tab) => {
  if (info.url && /^(file|https?):/.test(info.url)) {
    keepAlive();
  }
});
keepAlive();
let age = 0;
setInterval(() => console.log(`Age: ${++age}s`), 1000);

```

Example 7-9b background.js

To see this working:

- Install the extension and ensure all devtools windows are closed.
- Have at least one tab open that has a protocol of `http://`, `https://`, or `file://`
- Open the `chrome://serviceworker-internals` page. You will see the extension continue to log to the console indefinitely.

Drawbacks and Limitations

The main issue with this strategy is that it relies on a content script on the other end to complete the connection. If, for example, the user only has tabs open with `chrome://` protocols, the background script will have no eligible tab to connect to, and the service worker will not connect and eventually terminate. One solution to this issue is to force open an eligible tab using `tabs.create()`, but this may degrade the user's experience as they will see a new tab magically be created by the extension unexpectedly.

WarningOverall, this solution is a bit of a clever hack. At the time of publication, it does appear to force the service worker to persist indefinitely, but there is no guarantee that Chrome or any other browsers will continue to behave this way into the future. Your mileage may vary!

Summary

In this chapter, you learned all about how extension code can run in the background in the browser. You were shown the similarities and differences between service workers that run in a web page vs. in an extension, as well as all the important changes between background scripts in manifest v2 and v3. Next, we went through all the basics of inspecting and debugging background scripts. Finally, you were shown a variety of common patterns that developers use in background scripts, as well as a clever strategy for extending the lifetime of a service worker.

In the next chapter, you will learn about the two primary user interface components of extensions: the popup and options pages.